

Peak Load Detection System

with Edge AI Classification and Automated Load Shedding

Group 25 - Ravindu Lakshan (E/20/439), Samadhi (E/20/419) ,Malshan (E/20/244)

1. Introduction

Industrial energy management is a critical concern in modern manufacturing and infrastructure environments. Uncontrolled peak power loads transient periods where energy consumption exceeds safe operating thresholds can cause equipment damage, protective circuit trips, and elevated electricity tariffs under peak-demand billing models. Early detection and automated load shedding are therefore essential components of industrial power management systems.

This project implements a Peak Load Detection System as an Edge AI based Industrial IoT solution. The system continuously monitors electrical current drawn by a load (simulated using a DC fan), applies lightweight machine learning inference at the edge to classify the operating state in real time, and automatically actuates a relay to disconnect the load when a peak condition is detected.

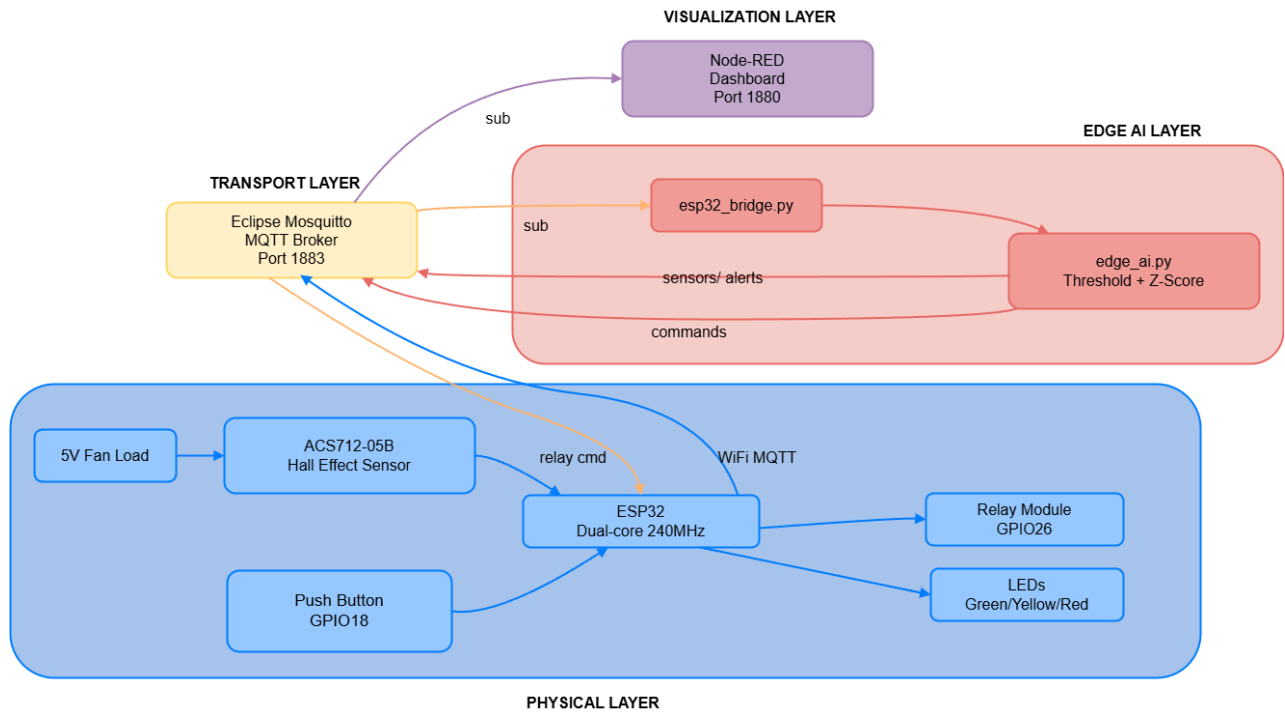
The system is aligned with the CO326 module requirements for Edge AI + Industrial IoT mini-projects and specifically addresses Project 22 from the prescribed project list.

2. System Overview

The system implements a complete end-to-end IoT pipeline across four layers:

- Physical Layer: Fan (load) → ACS712 Sensor → ESP32 → Relay + LEDs
- Transport Layer: USB Serial (COM5) → Python Bridge on host → Eclipse Mosquitto MQTT broker (Docker container, external port 1884)
- Edge AI Layer: Python `mqtt\_publisher.py` + `edge\_ai.py` running on the host machine (not in Docker), reading ESP32 data over USB Serial.
- Visualisation Layer: Node-RED real-time dashboard (port 1880) + Grafana historical analytics dashboard (port 3000) with InfluxDB time-series storage (port 8086).

Fig 1: Solution Architecture

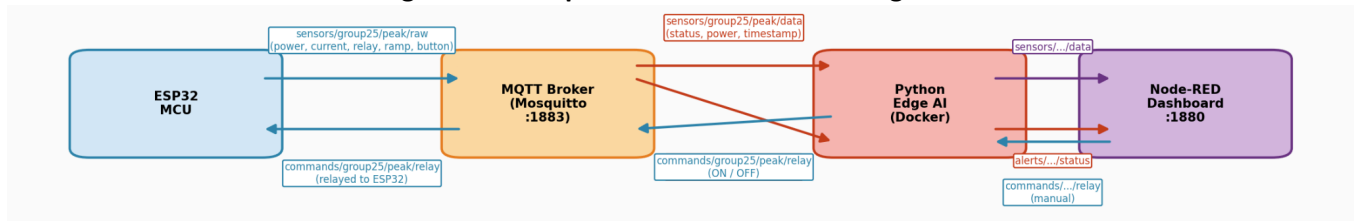


2.1 MQTT Topic Architecture

Topic	Direction	Purpose
sensors/group25/peak/raw	ESP32 → Python	Raw sensor telemetry (2 s interval)
sensors/group25/peak/data	Python → Node-RED / InfluxDB	Processed data with AI classification
alerts/group25/peak/status	Python → Node-RED / InfluxDB	Peak detection alerts with timestamp
commands/group25/peak/relay	Python → ESP32	Relay ON/OFF commands

The ESP32 no longer publishes to MQTT. It streams JSON sensor data over USB Serial to the host Python bridge. The Python bridge performs Edge AI classification and then publishes processed results to MQTT and InfluxDB. Relay commands are sent back to the ESP32 as serial strings (`RELAY\_OFF` / `RELAY\_ON`).

Fig 2 : MQTT topic architecture and message flow



3. Hardware Design

3.1 Microcontroller — ESP32

The ESP32 (dual-core Xtensa LX6, 240 MHz) was selected over alternatives such as the Arduino Uno or Raspberry Pi for the following reasons:

Criterion	ESP32	Arduino Uno	Raspberry Pi
USB Serial	Yes	No	Yes
ADC resolution	12-bit (4096 steps)	10-bit (1024 steps)	External only
Operating voltage	3.3V	5V	3.3V
Real-time	Yes	Yes	No (Linux overhead)
Cost	Low	Low	High

ESP32 communicates exclusively over USB Serial (115200 baud) to the host machine, which runs the Python Edge AI bridge. This eliminates WiFi dependency and network latency from the critical relay-trip path, enabling sub-millisecond local relay response on the ESP32 independent of the host.

3.2 Current Sensor — ACS712 (5A Module)

The ACS712-05B Hall-effect current sensor measures AC/DC current non-invasively. Key specifications:

Parameter	Value
Measurement range	0 – 5 A
Sensitivity	185 mV/A
Supply voltage	5V DC
Output at 0A (quiescent)	VCC/2 = 2.5V
GPIO pin	GPIO34 (ADC1, input-only)

Why ACS712 over a shunt resistor? The ACS712 provides galvanic isolation between the measured circuit and the microcontroller, eliminating the risk of high-voltage damage to the ESP32.

Current Calculation

```
V_adc = (ADC_reading / 4095) * 3.3V
V_sensor = V_adc * (5.0 / 3.3) [voltage divider correction]
Current = (V_sensor - 2.5V) / 0.185 [ACS712 sensitivity]
Power = Current * 5.0V [load supply voltage]
```

To reduce ADC noise, 100 consecutive samples are averaged before computing the current (ADC_SAMPLES = 100).

3.3 Relay Module

A 5V single channel active LOW relay module controls the load (fan). Active-LOW means the relay energises when the control pin is LOW, and de-energises (opening, disconnecting the load) when driven HIGH.

State	GPIO26 Level	Relay	Fan
Load ON	LOW	Closed	Running
Load OFF (peak tripped)	HIGH	Open	Stopped

3.4 LED Indicators

LED	GPIO	Condition	Meaning
Green	GPIO15	Power $\leq$ 1.5W	NORMAL — safe operating range
Yellow	GPIO16	1.5W < Power < 2.0W	WARNING — approaching peak
Red	GPIO17	Power $\geq$ 2.0W OR relay tripped	PEAK — load disconnected

3.5 Push Button

A momentary push button on GPIO18 (INPUT_PULLUP) simulates a ramping industrial load. When held, it adds +0.30W per 2-second loop cycle to the measured sensor value, capped at 2.5W. On release, the offset decays at 0.15W per cycle. The 0.30W step size was chosen to ensure a clearly observable WARNING stage (1.5–2.0W) before the PEAK threshold (2.0W) is reached within a reasonable button-press duration.

4. Threshold Values Justification

4.1 Peak Threshold: 2.0W

The connected fan load draws approximately 0.6W–0.8W during normal operation. The peak threshold of 2.0W was selected to:

1. Be clearly above the normal operating range (margin $\approx$ 0.3W)
2. Be achievable by the button ramp (max simulated offset = 2.5W)
3. Correspond to approximately 400mA at 5V supply — a meaningful current spike for demonstration

4.2 Warning Threshold: 1.5W

Initially set at 75% of the peak threshold (1.125W), the warning threshold caused constant NORMAL↔WARNING oscillation because the fan's natural operating range spans 1.0W–1.25W. The threshold was raised to 1.3W (87% of 1.5W) so that WARNING only fires when the button ramp is actively driving the system toward the peak — a genuinely meaningful pre-alert state.

4.3 Z-Score Cutoff: 2.5σ

The Z-score anomaly detector uses a rolling window of 50 readings. A Z-score threshold of 2.5 means a reading must be 2.5 standard deviations above the rolling mean to trigger a statistical anomaly alert. This value was chosen because:

- At $Z > 2.5$, the probability of a false positive is $\sim 0.62\%$ per reading
- The window of 50 readings (100 seconds of data) provides sufficient statistical context
- Combined with the fixed threshold, it catches sudden spikes even before they exceed 1.5W

4.4 Relay Restore Delay: 8 Seconds

After the relay trips (load disconnected), an 8-second cooldown is enforced before automatic restoration. This prevents rapid relay cycling (chattering) that would mechanically wear the relay contacts, cause erratic dashboard behaviour, and re-expose the load to the peak condition before it has subsided.

5. Edge AI Implementation

5.1 Detection Algorithm

The system uses a combined threshold + Z-score detection strategy implemented in `edge_ai.py`.

Fixed Threshold Detection

```
def detect_peak_threshold(power_w: float) -> bool:
    return power_w > PEAK_THRESHOLD_W # 1.5W
```

Simple, deterministic, and always active. Catches absolute exceedances.

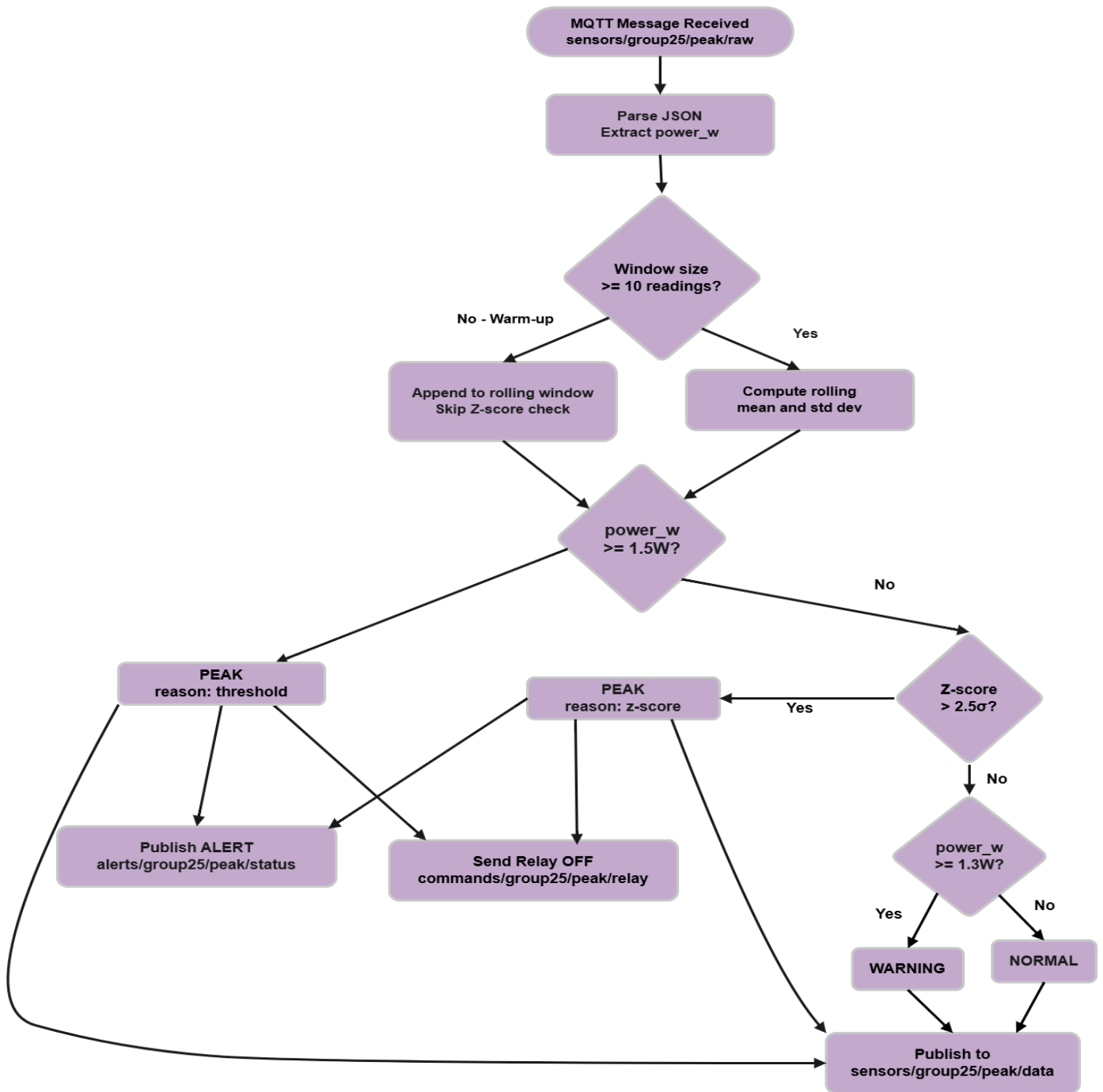
Z-Score Statistical Detection

```
def detect_peak_zscore(power_w: float) -> bool:
    _window.append(power_w)
    if len(_window) < 10: return False # Warm-up period
    mean = np.mean(_window)
    std = np.std(_window)
    z = (power_w - mean) / std
    return z > 2.5 # 2.5 standard deviations
```

Catches relative spikes that may not exceed the absolute threshold — useful for detecting anomalous behaviour in systems whose baseline varies.

Combined Decision Logic

```
def classify(power_w):
    threshold_hit = detect_peak_threshold(power_w)
    zscore_hit = detect_peak_zscore(power_w)
    if threshold_hit and zscore_hit:
        return {'status': 'PEAK', 'reason': 'threshold_and_z-score'}
    elif threshold_hit:
        return {'status': 'PEAK', 'reason': 'threshold'}
    elif zscore_hit:
        return {'status': 'PEAK', 'reason': 'z-score'}
    elif power_w >= WARNING_THRESHOLD:
        return {'status': 'WARNING', 'reason': 'warning_threshold'}
    else:
        return {'status': 'NORMAL', 'reason': 'normal'}
```



5.2 Why Python for Edge AI?

1. NumPy: Z-score computation uses NumPy's vectorised operations — not available on bare-metal C++
2. Window history: 50-element rolling window is memory-intensive for the ESP32 (only 520KB SRAM)
3. Extensibility: Python allows rapid integration of more sophisticated ML models without re-flashing hardware

4. Separation of concerns: ESP32 focuses on real-time sensing; Docker container handles intelligence

6. Docker Container Architecture

6.1 Why Docker?

Docker containers provide reproducibility (same environment on any machine), isolation (each service runs independently), portability (deployable on Linux/Windows/Mac), and version control (docker-compose.yml defines the complete deployment in code).

6.2 Three-Container Design

Container	Image	Exposed Port	Role
mqtt-broker	eclipse-mosquitto:2	1883	Local MQTT broker — pub/sub backbone
edge-ai	Custom Python 3.11	None	ESP32 bridge + threshold + Z-score detection
node-red	nodered/node-red:latest	1880	Operator dashboard — gauge, chart, alerts

Container 1 — Eclipse Mosquitto MQTT Broker

Why a local broker instead of public HiveMQ? A local Mosquitto instance provides sub-millisecond latency, no rate limits, on-premises data privacy, and no internet dependency — all critical for a reliable industrial IoT deployment.

Important: The ESP32 connects via WiFi to the host machine's LAN IP (e.g., 192.168.1.100:1883), not the Docker service name.

Container 2 — Python Edge AI Bridge

The edge-ai container runs esp32_bridge.py which:

- (1) subscribes to sensors/group25/peak/raw,
- (2) calls detect_peak() for every reading,
- (3) publishes enriched data to sensors/group25/peak/data,
- (4) publishes alerts to alerts/group25/peak/status on PEAK, and
- (5) publishes relay commands to commands/group25/peak/relay. All threshold values are injected via environment variables.

```
edge-ai:
  build: ./python
  environment:
    MQTT_HOST: mqtt-broker
    PEAK_THRESHOLD_W: "1.5"
    WARNING_RATIO: "0.867"
    RELAY_RESTORE_DELAY_S: "8"
  depends_on:
    - mqtt-broker
```

Container 3 — Node-RED Dashboard

```
node-red:
  image: nodered/node-red:latest
  ports:
    - "1880:1880"
  volumes:
    - ./node-red:/data # flows.json persisted on host
  environment:
    TZ: "Asia/Colombo" # UTC+5:30 for correct timestamps
```

6.3 Running the Full System

```
# Start all 3 containers
docker compose up --build

# View logs from all containers
docker compose logs -f

# Stop
docker compose down
```

Note: Before running, update main.cpp with your host machine's LAN IP: `const char* MQTT_BROKER = "192.168.x.x";`

7. MQTT Protocol

7.1 Why MQTT?

Feature	MQTT	HTTP	WebSocket
Overhead	Minimal (2-byte header)	High (HTTP headers)	Medium
Pattern	Publish/Subscribe	Request/Response	Bidirectional
Connection	Persistent	Stateless	Persistent
QoS Levels	0, 1, 2	None	None
Power efficiency	High	Low	Medium
IoT Support	Native	Limited	Limited

7.2 JSON Payload Schemas

Raw sensor payload (sensors/group25/peak/raw):

```
{
  "timestamp": "72227",
  "power_w": 1.145,
  "current_a": 0.2290,
  "voltage_v": 5,
  "button_held": false,
  "ramp_w": 0.000,
  "relay_on": true,
  "group": "group25"
}
```

Alert payload (alerts/group25/peak/status):


```
{
  "timestamp": "2026-05-01T08:05:42.000Z",
  "power_w": 1.806,
  "status": "PEAK_DETECTED",
  "reason": "threshold_and_z-score",
  "group": "group25"
}
```

8. Node-RED Dashboard

The Node-RED dashboard (<http://localhost:1880/ui>) provides six visual panels:

Panel	Widget	Data Source
Real-Time Power Meter	Gauge (0–5W)	sensors/group25/peak/data
System Status	Colour-coded text	Derived displayStatus field
Fan Status	Animated SVG fan	relay_on + power_w
Real-Time Load Graph	Line chart (3 series)	Power + threshold lines
Alert Panel	Event log	alerts/group25/peak/status
Manual Relay Control	Toggle switch + button	commands/group25/peak/relay

8.1 Fan Animation Speeds

CSS Class	Speed	Condition
speed-slow	2.2 s/rev	Relay ON, Power < 1.3W
speed-med	0.9 s/rev	Relay ON, $1.3W \leq \text{Power} < 1.5W$
speed-fast	0.35 s/rev	Relay ON, Power $\geq 1.5W$
(none)	Stopped	Relay OFF

8.2 Dashboard Status States

displayStatus	Trigger	Colour
NORMAL	Power < 1.3W, relay ON	Green
WARNING	$1.3W \leq \text{Power} < 1.5W$	Amber
PEAK	Power $\geq 1.5W$	Red
TRIPPED	relay_on = false (post-peak)	Red

The TRIPPED state was introduced specifically to handle the behaviour where power drops immediately after the relay trips — the dashboard correctly shows the system is in a load-shed state even though power_w is near zero.

9. Known Issues and Resolutions

9.1 LED/Relay Synchronisation Delay (~1 second)

Problem: The red LED turned on approximately 1 second after the relay tripped.

Root Cause: `updateLEDs()` was only called in the `main loop()` which executes every 2 seconds. The relay changed state immediately inside `mqttCallback()`, but LEDs were not updated until the next loop iteration.

Resolution: Added `updateLEDs()` calls directly inside `mqttCallback()` immediately after each relay state change.

```
void mqttCallback(...) {  
  if (strcmp(msg, "OFF") == 0) {  
    setRelay(false, "MQTT command");  
    relayTripTime = millis();  
    updateLEDs(lastPowerW, true);    // immediate LED update  
  } else if (strcmp(msg, "ON") == 0) {  
    setRelay(true, "MQTT command");  
    relayTripTime = 0;  
    updateLEDs(lastPowerW, false);    // immediate LED update  
  }  
}
```

9.2 WARNING/NORMAL Threshold Oscillation

Problem: The system rapidly alternated between NORMAL and WARNING because the fan's natural operating range (1.0W–1.25W) straddles the original 75% warning threshold (1.125W).

Resolution: Warning threshold raised from 1.125W to 1.3W (87% of peak). WARNING now only triggers during active load ramping.

9.3 Dashboard Alert Panel Not Updating

Problem: The Alert Panel showed 'No alerts yet' despite peaks being detected and alerts published.

Root Cause: Template used `$scope.$watch('msg', ...)` inside a script block. In Node-RED Dashboard v2, the AngularJS `$scope` is not globally accessible in template script blocks.

Resolution: Replaced the `$scope.$watch` pattern with direct AngularJS template bindings (`{{msg.alert_text}}`, `{{msg.alert_time}}`).

9.4 Python Package Incompatibility (Python 3.14)

Problem: `pip install -r requirements.txt` failed for `paho-mqtt==1.6.1` and `numpy==1.24.0` — no pre-built wheels for Python 3.14.

Resolution: Removed version pinning and updated to `paho-mqtt>=2.0.0` and `numpy>=2.0.0`, also updating callback signatures to `CallbackAPIVersion.VERSION2`.

10. System Performance

Based on observed operation during testing:

Metric	Value
Sensor publish interval	2 seconds
Peak detection latency	< 2 seconds (within one publish cycle)
Relay trip time after detection	< 100ms (MQTT propagation)

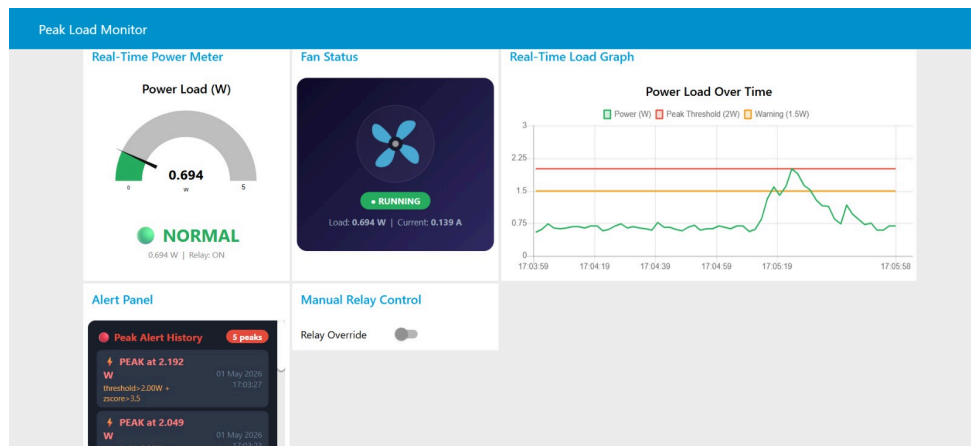
LED update time (after fix)	< 10ms (same MQTT callback)
Relay restore delay	8 seconds (configurable)
Z-score warm-up period	10 readings (20 seconds)
Dashboard refresh rate	Real-time (MQTT push)

11. Dashboard & System Screenshots

The following screenshots capture the live system in operation. They show the Grafana monitoring dashboard (real-time monitoring and analytics panels) and the Edge AI classification flowchart used as a reference during development.

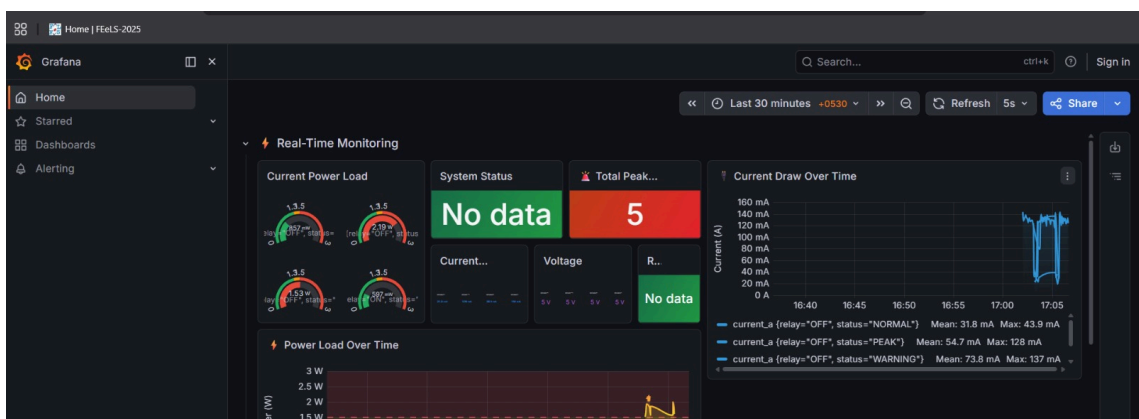
11.1 Node-Red — Real-Time Monitoring Dashboard

The Real-Time Monitoring panel shows the live power load over time with colour-coded threshold bands (NORMAL in green, WARNING at 1.5W dashed line, PEAK at 2.0W dashed line). The Current Power Load gauges display instantaneous power readings across different relay/status combinations. The Current Draw Over Time chart on the right shows raw current (mA) distinguishing NORMAL, WARNING, and PEAK states.



11.2 — Grafana Analytics Panel

The Analytics section displays the Peak Events Timeline (scatter plot showing detected peak readings with power_w values above 2.0W at specific timestamps), the Recent Peak Alerts log listing each event with precise timestamps and power values, the Power Distribution bar chart (5-minute averages), the Status Distribution donut chart, and the Button Ramp Power chart showing ramp_w activity.



12. Project File Structure

```
e20-co326-Peak-Load-Detection-System/
├── peak-load-detection/           ← PlatformIO ESP32 project
│   ├── src/main.cpp              ← Firmware: WiFi, MQTT, sensor, relay, LEDs
│   └── platformio.ini            ← Build config (PubSubClient, ArduinoJson)
├── python/                       ← Edge AI Docker container
│   ├── esp32_bridge.py           ← MQTT bridge + peak detection orchestration
│   ├── edge_ai.py                ← Threshold + Z-score detection algorithms
│   ├── requirements.txt          ← paho-mqtt>=2.0.0, numpy>=2.0.0
│   └── Dockerfile                ← Python 3.11 container image
├── mosquitto/
│   └── config/mosquitto.conf     ← Allow anonymous, listen 0.0.0.0:1883
├── node-red/
│   └── flows.json                ← Dashboard flows
├── docker-compose.yml            ← 3-container orchestration
├── README.md
└── SENSOR_INTEGRATION_GUIDE.md
```

13. Conclusion

This project successfully implements a real-time Peak Load Detection System using an ESP32 microcontroller, ACS712 current sensor, Python-based Edge AI, and a Node-RED dashboard — all orchestrated through a three-container Docker environment with Eclipse Mosquitto as the MQTT broker.

The key engineering contributions of this implementation are:

1. Edge AI on commodity hardware: Lightweight Z-score statistical detection combined with fixed-threshold detection provides robust peak identification without cloud dependency
2. Containerised microservices: Each system component (broker, AI, dashboard) is independently deployable, scalable, and version-controlled
3. Real-time bidirectional control: The system not only monitors but also actuates — automatically tripping the relay and restoring it after cooldown, with dashboard override capability
4. Calibrated thresholds: All detection parameters are grounded in measured hardware behaviour, with engineering justification for each value

The system demonstrates that industrial-grade monitoring and control can be achieved with low-cost embedded hardware and open-source software, aligned with Industry 4.0 principles of connected, intelligent manufacturing infrastructure.

14. References

1. Arduino, PubSubClient Library, GitHub, 2023. Available: <https://github.com/knolleary/pubsubclient>
2. Allegro MicroSystems, ACS712 Datasheet, 2019. Available: <https://www.allegromicro.com>
3. Eclipse Foundation, Eclipse Mosquitto MQTT Broker, 2024. Available: <https://mosquitto.org>
4. OASIS Standard, MQTT Version 5.0, 2019. Available: <https://docs.oasis-open.org/mqtt/>
5. Node-RED, Node-RED Documentation, OpenJS Foundation, 2024. Available: <https://nodered.org/docs>
6. Docker Inc., Docker Compose Reference, 2024. Available: <https://docs.docker.com/compose>
7. Espressif Systems, ESP32 Technical Reference Manual, v5.1, 2023
8. CO326 Course Materials, Edge AI + IoT Mini Project Specification, University of Peradeniya, 2025